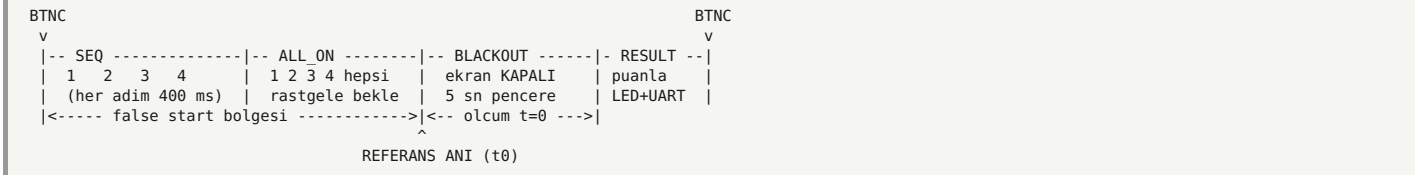


Basys3 Refleks Oyunu — Grup Kılavuzu

Nazik Ormanlar · BİL 264/265 · Kim ne yazacak, neye karar verilecek, nasıl yazılacak

1. Projenin tamamı tek sayfada

Basys3 üzerinde 2–4 oyunculu bir refleks yarışı. Switch'lerle ayar yapılır, BTNC ile tur başlar. 7-segment'in dört basamağı sırayla yanar, hepsi yanık kalır, **LFSR'den gelen rastgele bir süre** geçer ve hepsi **aynı anda söner**. O an sıfır noktasıdır; oyuncular butonlarına basar, süreler 1 ms çözünürlükle ölçülür. Erken basan (false start) ve 5 sn içinde basmayan (timeout) 0 puan alır. Sıralamaya göre 4/3/2/1 puan verilir, LED'lerde gösterilir, UART'tan terminale rapor yazılır.



2. Mevcut durum

Durum	Dosyalar
Bitti ve doğrulandı	timebase.v, debouncer.v — testbench'leriyle birlikte simülasyonda çalıştı
Hazır (iskelet)	12 modül + 7 testbench, portları tanımlı, gövdeleri boş
Hazır	Basys3_Refleks.xdc — pinler kart kılavuzundan doğrulandı
Yapılacak	12 modülün gövdesi, 7 testbench, top.v bağlantıları, bitstream

Bitmiş iki modül **Katman 0**: hiçbir şeye bağlı değiller, ama neredeyse her şey onlara bağlı. Bundan sonra yazılacak her modül bu ikisinin üstüne oturuyor.

3. Kod yazmadan önce karar verilecekler

Bunları **ilk 30 dakikada**, dördünüz birlikte karara bağlayın ve top.v'nin başına comment olarak yazın. Kodda karar vermeye kalkarsanız entegrasyon günü çakışsınız.

3.1 Kesin karar gerektirenler

#	Konu	Öneri
K1	Switch haritası. XDC'de öneri yazılı: SW15 reset, SW14 eleme, SW13 zorluk, SW12:11 oyuncu sayısı, SW3:0 tur sayısı.	Olduğu gibi kabul edin; XDC ve top.v zaten buna göre.
K2	SW12:11 = 11 ne olacak? 00→2, 01→3, 10→4 belli; 11 tanımsız.	11 → 4 oyuncu sayın. Alternatif: geçersiz kabul edip oyunu başlatmamak.
K3	İsimlendirme dili. Şu an karma: vurus_1ms Türkçe, disp_sel İngilizce.	Sinyaller Türkçe, modül isimleri ve teknik terimler İngilizce. Bir isim değişirse gruba haber.
K4	SEQ animasyon adım süresi. Şartname belirtmiyor.	400 ms. Toplam 1,6 sn sürer, oyuncuya hazırlanma hissi verir.
K5	Basamaklar birikerek mi yansın? 1 → 12 → 123 → 1234 (F1 ışıkları gibi) veya tek tek 1 → 2 → 3 → 4.	Birikerek. Videodaki mantık bu ve blackout'un etkisi daha güçlü olur. Hocaya doğrulatin.
K6	Konfigürasyon sırasında ekran ne gösterecek?	Ayarları göstermek zorunlu değil ama demoda iyi durur: oyuncu sayısı + tur sayısı.
K7	Berberlikte LED. İki oyuncu aynı ms'de basarsa ikisi de birinci; ikisine de 4 LED mi?	Evet. Puan = LED sayısı kuralı zaten bunu doğal veriyor.
K8	Elenen oyuncunun LED'leri. Şartname belirtmiyor.	Sönük kalsın (0 puan → 0 LED). Ekstra kod gerekmez.
K9	2 veya 3 oyuncuda puanlama. Birinci 4 puan mı, yoksa oyuncu sayısına göre mi?	Her zaman 4/3/2/1. Şartname sıraya göre diyor, oyuncu sayısına göre demiyor.
K10	UART satır sonu. \n mi \r\n mi?	\r\n. PuTTY ve Tera Term'de satırlar düzgün görünür.
K11	Erken bitiş anı. Eleme modunda tek oyuncu kalınca hemen mi, tur bitince mi?	Tur bitip rapor gönderildikten sonra. Yarım kalan tur kafa karıştırır.
K12	Oyun bitince 7-segment.	Kazananın numarasını göster (örn. P 3) veya tamamen söndür. Basit olanı seçin.

3.2 Hocaya sorulacaklar

- K5** — basamaklar birikerek mi yoksa tek tek mi yanmalı?
- Eleme modu **kapalıyken** false start yapan oyuncu, aynı turda blackout'tan sonra tekrar basarsa süresi sayılsın mı? (Öneri: hayır, o tur için 0 puan kesin.)
- Tur sayısı 4 bit ile giriliyor ve 0000→1 tur, 1111→16 tur. Yani **N+1**. Doğrulatin.

4. İş bölümü

Kişi	Modüller	Neden bu grup
A — entegratör	top, XDC bakımı, seg7_driver, led_ctrl, kart üstü debug	Entegrasyon ve donanım debug'ı seri iştir, tek kişi sahiplenmeli. Kendi modülleri hafif olmalı ki entegrasyona vakti kalsın.
B	game_fsm	Tek beyin işi. İkiye bölünürse iki kişi birbirinin durumunu bozar. En son yazılır, alttakiler hazır olunca çok daha kolaydır.
C	lfsr16, delay_gen, reaction_capture, scoring, score_accum	Hepsi FSM'in sinyallerini tüketen "hesap" blokları, birbirine yakın ve zincirleme test edilebilir.
D	uart_tx, bin2bcd, uart_reporter	UART bağımsız bir alt sistem. Kimseyi beklemeden ilerler, ama en çok hafife alınan kısımdır.

Kritik: B ve D en uzun süren işlere sahip. A ve C kendi modüllerini bitirdiklerinde onlara yardım etmeye hazır olsun. "Ben bitirdim" demek proje bitti demek değil.

5. Arayüz sözleşmesi

Bu tablo **tek gerçek kaynak**. Kod bununla çelişiyorsa kod yanlıştır.

Üreten	Sinyal	Bit	Tüketen
timebase	vurus_1ms	1 (puls)	herkes
timebase	disp_sel	2	seg7_driver
debouncer x5	btn_puls	1 (puls)	game_fsm (BTNC), reaction_capture (P1-P4)
lfsr16	deger	16	delay_gen
delay_gen	bekleme_ms	13	game_fsm
game_fsm	silahli / pencere / t0	1 / 1 / 1(puls)	reaction_capture
game_fsm	basamak_en, d0..d3	4, 4x4	seg7_driver
game_fsm	oyuncu_maske, eleme_modu, zor_mod	4,1,1	score_accum, delay_gen
game_fsm	tur_no, tur_sonu, oyun_bitti, uart_basla	5,1,1,1	score_accum, led_ctrl, uart_reporter
reaction_capture	sure0..sure3	13 er	scoring, uart_reporter
reaction_capture	gecerli, yanlis_baslangic, zaman_asimi	4 er	scoring, score_accum, uart_reporter
reaction_capture	tur_hazir	1	game_fsm
scoring	puan0..puan3	3 er	score_accum, led_ctrl
score_accum	toplaml0..toplaml3, yasayan, kazanan, beraberlik	7er,4,2,1	uart_reporter, led_ctrl, game_fsm
uart_reporter	tx_gonder, tx_veri	1, 8	uart_tx
uart_tx	tx	1	RsTx pini (A18)

aktif = oyuncu_maske & yasayan — top.v içinde hesaplanır, reaction_capture'a verilir.

6. Herkesin uyacağı kurallar

Kural	Neden
Sıralı bloklarda <=, kombinasyonel bloklarda =	Karıştırsan simülasyon ile donanım farklı davranır; hatayı bulmak günler alır.
Reset senkron : always @(posedge clk) if (rst)	7-serisi FF'lerin özel senkron reset girişi var; asenkronda reset recovery problemi çıkar.
Kombinasyonel blokta her yola varsayılan değer	Bir dalda atama yapmazsan sentezleyici latch üretir. FPGA'da latch istemezsin.
always @(*) yaz, duyarlılık listesini elle yazma	Bir sinyali unutursan simülasyon ile sentez tutmaz.
Reset'i olmayan ve kendini besleyen sayaçlara = 0 başlangıç değeri ver	Simülasyonda X + 1 = X; sayaç sonsuza kadar X'te kilitlenir. Donanımda sorun çıkmaz, sadece simülasyonda.
Zamanı vurus_1ms ile say, ham çevrim sayma	Ortak metronom; ölçümler birbirine göre kaymaz ve MS_DIV ile hepsi birden hızlanır.
Yeni saat üretme , enable pulsu kullan	Fabric'te üretilen saat global saat ağına girmez, skew olur, Vivado zamanlama analizi yapamaz.
Testbench'te MS_DIV / CLKS_PER_BIT küçült	Gerçek değerle 5 saniyelik pencere 500 milyon saat kenarı eder.
Uyaranları negedge 'de değiştir	Posedge'de değiştirirsen DUT'un aynı kenardaki örneklemesiyle yarışır; sonuç tanımsızdır.
UART metninde Türkçe karakter yok	Terminal kodlaması yüzünden bozuk görünür.

7. Modül modül ne yazılacak

7.1 lfsr16

C

Ne yapar: Sözde-rastgele 16 bitlik sayı üretir. Resetten itibaren **kesintisiz** döner, turlar arasında durdurulmaz.

Yazılacak

- Geri besleme biti: `deger[15] ^ deger[13] ^ deger[12] ^ deger[10]` — 16 bit için maksimum uzunluk tapları.
- `rst` → `deger <= SEED` (varsayılan 16'hACE1).
- Değilse → `deger <= {deger[14:0], geri_besleme}` (sola kaydır, geri beslemeyi en sağa koy).

Anlaman gereken: Asıl rastgelelik LFSR'de değil, **oyuncunun BTNC'ye bastığı anın öngörülemezliğinde**. LFSR sadece sürekli dönen bir zar; sen o anda üstündeki sayıyı okuyorsun. Şartnamenin "durdurmayın" demesinin sebebi bu. Tüm bitler 0 olursa geri besleme de 0 üretir ve LFSR sonsuza kadar kilitlenir — SEED bu yüzden sıfır olamaz.

Testbench: `lfsr16_tb` — dalga formuna bak. `deger` hiç 0 olmamalı; SEED'e dönmesi 65535 çevrim sürmeli.

7.2 delay_gen

C

Ne yapar: LFSR'nin ham sayısını istenen zaman aralığına ölçekler. Tamamen kombinasyonel, saat kullanmaz.

Yazılacak

```
r      = lfsr_deger[11:0]          -> 0..4095
t_min  = zor_mod ? 500 : 2000
aralik = zor_mod ? 4500 : 3000
bekleme_ms = t_min + ((r * aralik) >> 12)
```

Neden bölme yok: Sentezde bölücü çok pahalı. Sabitle çarpıp sağa kaydırmak matematiksel olarak aynı işi görür ve Vivado bunu DSP bloğuna atar. Genel kural: donanımda / ve % görürsen dur ve düşün.

Testbench: Aynı testbench şart değil; `lfsr16_tb`'ye ekleyip birkaç LFSR değerinde çıkan süreyi kontrol edin. Uç durumlar: `r=0` → `t_min`, `r=4095` → `t_min+aralik`'a çok yakın.

7.3 seg7_driver

A

Ne yapar: 4 basamaklı ortak anotlu ekranı sürer. Basamaklar segment hatlarını paylaştığı için aynı anda farklı rakam gösterilemez; çözüm çok hızlı sırayla birer basamak yakmak.

Yazılacak (tek always @(*))

- `disp_sel`'e göre `d0..d3`'ten birini seç.
- `an = 4'b1111` (hepsi kapalı). `basamak_en[disp_sel]` 1 ise sadece `an[disp_sel] = 0` yap. **Aktif-düşük**.
- Seçilen rakamı segment desenine çevir (case). Desenler iskelet dosyasında yazılı.

En sık yapılan hata: Donanımsal taramayı (saniyede ~1500 kez, gözle görünmez) oyunun 400 ms'lik animasyonu ile karıştırmak. Tarama **her zaman** çalışır; animasyon sadece `basamak_en` maskesini değiştirir. Bu ayrımı kuramayan gruplar burada takılıyor.

Testbench: `seg7_driver_tb` — saat bile gerekmez. `disp_sel`'i 0,1,2,3 yap, her birinde `an` ve `seg`'i kontrol et. Sonra `basamak_en = 4'b0011` yapıp üst iki basamağın söndüğünü gör.

7.4 reaction_capture

C

Ne yapar: Kronometre + hakem. `t0` anından itibaren ms sayar, her oyuncunun basış anını yakalar, false start ve timeout kararını verir.

Yazılacak

- `rst` veya `t0` → `ms <= 0`, üç bayrak setini (`gecerli`, `yanlis_baslangic`, `zaman_asimi`) temizle.
- `pencere && vurulms` → `ms <= ms + 1`.
- Her oyuncu `i` için, `aktif[i] && basis[i]` ise:
 - `silahlili` → `yanlis_baslangic[i] <= 1`
 - `pencere && !gecerli[i] && !yanlis_baslangic[i]` → `sure_i <= ms`, `gecerli[i] <= 1`
- `pencere && ms == 5000` → `zaman_asimi <= aktif & ~gecerli & ~yanlis_baslangic`.
- `tur_hazir = 5000` ms doldu **veya** aktif oyuncuların hepsi çözüldü.

Bedava kazanç: `gecerli[i]` bayrağı ikinci basışı da engelliyor — oyuncu butonu basılı tutsa bile tek ölçüm alınır. Ayrı kilit mekanizması kurmayın.

Testbench: `reaction_capture_tb` — **burada \$display gerekli**. 4 süre ve 3 bayrak setini dalga formunda gözle doğrulamak yorucu ve hataya açık. Dört senaryo: false start, normal (farklı süreler), beraberlik (aynı çevrimde iki basış), timeout.

7.5 scoring

C

Ne yapar: Süreleri sıraya dizip puana çevirir. Tamamen kombinasyonel.

Sıralama algoritması yazmanıza gerek yok. Tek fikir:

```
sira[i] = "benden KESİNLİKLE daha hızlı kaç geçerli oyuncu var"
puan[i] = gecerli[i] ? (4 - sıra[i]) : 0
```

Bu tek satır iki şeyi bedava veriyor: **(1)** aynı ms'de basanların cevabı aynı çıkar, yani beraberlik kuralı ekstra kod istemez. **(2)** puan sayısı = yanacak LED sayısı, led_ctrl'de tekrar hesap yok.

Sıra	sıra[i]	puan	LED
1.	0	4	4
2.	1	3	3
3.	2	2	2
4.	3	1	1
False start / timeout	—	0	0

Testbench: scoring_tb — \$display gerekli. Saat bile gerekmez. Dört senaryo: normal, beraberlik, geçersiz basış, 2 oyunculu.

7.6 score_accum

C

Ne yapar: Puan defteri + eleme kararı.

Yazılacak

- rst → toplamlar 0, yasayan <= oyuncu_maske.
- tur_sonu → toplam_i <= toplam_i + puan_i. Eleme modu açık ve oyuncu false start/timeout yaptıysa yasayan[i] <= 0.
- Kombinasyonel: en büyük toplamı bul → kazanan. Birden fazla oyuncu aynı en büyük değerdeyse beraberlik = 1.

Genişlik: 16 tur × 4 puan = 64, yani 7 bit. yasayan içinde tek 1 kalması game_fsm'in erken bitiş koşulu.

Testbench: Aynı testbench şart değil, scoring_tb'ye eklenebilir. Kontrol edilecek: birikme doğru mu, eleme modu kapalıyken kimse elenmiyor mu.

7.7 led_ctrl

A

Ne yapar: 16 LED'i sürer. Oyuncu 1: LED0-3, Oyuncu 2: LED4-7, Oyuncu 3: LED8-11, Oyuncu 4: LED12-15.

Yazılacak

- termo fonksiyonu — puanı termometre desenine çevirir: 4→1111, 3→0111, 2→0011, 1→0001, 0→0000. İpucu: 4'b1111 >> (4-p).
- oyun_bitti ise → tüm LED'ler 0, sadece kazananın 4 biti 1.
- Değilse → her oyuncunun 4 bitine termo(puan_i).

Testbench: Gerekmez, dalga formunda ya da kartta gözle doğrulanır.

7.8 bin2bcd

D

Ne yapar: İkilik sayıyı 4 haneli BCD'ye çevirir. 1342 ikilik bir bit yığınıdır; terminale "1342" yazmak için her basamağı ayrı elde etmen gerekir.

Yazılacak (double dabble)

- bcd = 0
- i = 12'den 0'a doğru: her 4 bitlik hane 4'ten büyükse o haneye 3 ekle; sonra bcd = {bcd[14:0], ikili[i]}

Bölme kullanmaz, sadece kaydırma ve toplama. ASCII'ye çevirmek için her haneye 8'h30 ekle.

Testbench: bin2bcd_tb — \$display ile kolay. Test değerleri: 0, 7, 42, 999, 1342, 5000.

7.9 uart_tx

D

Ne yapar: Tek bir baytı seri hatta bit bit gönderir. 9600 baud, 8N1 = 1 start biti (0) + 8 veri biti (LSB önce) + 1 stop biti (1). Bit süresi = 100.000.000 / 9600 = **10417 çevrim**.

Yazılacak — 4 durumlu FSM

Durum	Ne yapar
BOS	tx=1, mesgul=0. gonder gelirse veriyi kaydırma'ya al, tx=0 (start biti), mesgul=1, START'a geç.
START	CLKS_PER_BIT çevrim say, sonra ilk veri bitini tx'e koy.
VERI	Her CLKS_PER_BIT'te bir sonraki biti tx'e koy (toplam 8).
STOP	tx=1, bir bit süresi bekle, BOS'a dön.

Önce bunu doğrula. Metin üretmeye geçmeden önce sonsuz döngüde tek bir 'A' (8'h41) gönderip terminalde gör (PuTTY / Tera Term, doğru COM portu, 9600-8-N-1). Çalışmadan üstüne bir şey inşa etme — yoksa hatanın baud'da mı formatlamada mı olduğunu ayıramazsın.

Pin tuzağı: Kart kılavuzundaki TXD/RXD isimleri **bilgisayarın bakış açısına göre**. B18 = PC'nin gönderdiği hat = FPGA'nın *alıcı* ucu. Senin çıkışın **A18**. Ters bağlarsan terminalde hiçbir şey görmezsin ve saatlerce baud'la uğraşırsın.

Testbench: uart_tx_tb — CLKS_PER_BIT'i 8 gibi küçült. Kontrol: start biti 0 mu, 8 veri biti LSB-önce mi, stop biti 1 mi, mesgul doğru zamanlarda inip çıkıyor mu.

7.10 uart_reporter

D

Ne yapar: "TUR 3 / P1 342 ms / PUAN 4" metnini bayt bayt üretilip uart_tx'e sıraya koyar.

Önerilen yaklaşım

- Bir `adim` sayacı tut, her adımda **tek bayt** üret.
- `basla` gelince `adim <= 0`, `calisiyor <= 1`.
- `calisiyor && !tx_mesgul` ise: `adim`'a karşılık gelen baytı `tx_veri`'ye koy, `tx_gonder`'i 1 çevrim yükselt, `adim`'ı artır.
- Son adımda `calisiyor <= 0`, `bitti <= 1`.

Sabit metinleri `case` içine göm, sayıları `bin2bcd`'den gelen hanelere `8'h30` ekleyerek ASCII yap.

```
TUR 03
P1 0342 ms PUAN 4
P2 0410 ms PUAN 3
P3 FALSESTART PUAN 0
P4 TIMEOUT PUAN 0
TOPLAM P1=12 P2=09 P3=04 P4=07

KAZANAN P1 (18 PUAN) veya BERABERLIK P1=18 P3=18
```

Bu modülü son güne bırakmayın. Mantiğı zor değil ama çok fazla durum var. Projede en çok hafife alınan ve en çok yetismeyen parça budur. Sabit genişlikli format kullanın — hem kod hem okuma kolaylaşır.

7.11 game_fsm

B

Ne yapar: Oyunun beyni. Diğer modüller hesap makinesi, bu modül yönetmen. Dışarıya yaydığı **üç ana sinyalden** bütün oyun kuralları çıkar:

Sinyal	Anlamı	Sonucu
silahli	Basmak yasak (SEQ + ALL_ON)	Basan → false start
t0	Blackout anı, tek çevrimlik puls	Kronometre başlar
pencere	Basmak serbest (5 sn)	Basmayan → timeout

Durum tablosu

Durum	Yaptığı	Çıkış koşulu
S_CONFIG	Switch'leri oku, ekranda ayarları göster	btnc_puls → ayarları LATCH'le
S_ARM	Ekran kapalı, tur başlamayı bekle	btnc_puls → lfsr_deger'i örnekle, hedef <= bekleme_ms
S_SEQ	basamak_en: 0001→0011→0111→1111, her adım 400 ms. silahli=1	4 adım bitince
S_ALL_ON	basamak_en=1111, silahli=1	zaman == hedef → t0 pulsu
S_BLACKOUT	basamak_en=0000, pencere=1	tur_hazir → tur_sonu pulsu + uart_basla
S_RESULT	Raporun bitmesini bekle	rapor_bitti → son tur mu / tek kişi mi kaldı?
S_NEXT	Sonraki turu bekle	btnc_puls → tur_no++, S_ARM
S_OVER	oyun_bitti=1, final UART mesajı	Sadece rst çıkarır

Gösterilecek rakamlar: tur 1 → 1 2 3 4, tur 2 → 5 6 7 8, tur 3 → 1 2 3 4... İpucu: taban = tur_no[0] ? 5 : 1, sonra d0=taban, d1=taban+1, ...

Erken bitiş: Eleme modu açıkken her tur sonunda yasayan maskesindeki 1'leri say; 1 kaldıysa S_OVER'a geç.

Yazım tavsiyesi: Bu kadar durum ve çıkış varken tek always bloğu okunmaz hale gelir. Üç bloğa ayır: (1) durum kaydı, (2) sonraki durum mantığı always @(*), (3) çıkış mantığı. Kombinasyonel bloklarda her yola varsayılan değer vermeyi unutma, yoksa latch üretirsin.

Testbench: Kendi başına test etmek zor; top_tb ile birlikte doğrulanır. Ama ilk aşamada sahte (elle sürülen) girişlerle durum geçişlerini izlemek çok zaman kazandırır.

7.12 top

A

Ne yapar: Kendi mantığı yok, sadece bağlantı. Şartnamedeki "switch ve LED eşlemesi" comment'i bu dosyanın başına gelir.

Dikkat edilecekler

- Port isimleri XDC ile **birebir** aynı olmalı: clk, sw, btnC/U/L/R/D, led, seg, an, dp, RsTx. Değiştirirseniz XDC'yi de güncelleyin.
- dp'yi 1'b1'e bağlayın (aktif-düşük, yoksa nokta sürekli yanar).
- 5 adet debouncer örnekleyin: BTNC + 4 oyuncu butonu.
- basis[3:0] = {p4_puls, p3_puls, p2_puls, p1_puls} — BTNU=P1, BTNL=P2, BTNR=P3, BTND=P4.
- aktif = oyuncu_maske & yasayan
- MS_DIV ve CLKS_PER_BIT'i parametre olarak taşıyın ki top_tb küçültebilsin.

8. Yazma sırası ve kilometre taşları

#	Adım	Bitince ne görürsünüz
0	Arayüzleri dondurun (bölüm 3 + 5)	Kimse kimsenin portunu bozmuyor
1	<code>lfsr16 + delay_gen</code>	Simülasyonda her turda farklı bekleme süresi
2	<code>seg7_driver + top</code> 'a bağla	Kartta "1234" yanıyor — ilk görsel kilometre taşı
3	<code>game_fsm</code> iskeleti (CONFIG→ARM→SEQ→ALL_ON→BLACKOUT)	BTNC ile animasyon ve blackout çalışıyor
4	<code>reaction_capture + scoring + led_ctrl</code>	Oyun oynanabilir — "çalışan proje" çizgisi burası
5	<code>score_accum</code> , false start, timeout, eleme, erken bitiş	Şartnamedeki tüm kurallar işliyor
6	<code>uart_tx</code> (önce tek 'A') → <code>bin2bcd</code> → <code>uart_reporter</code>	Terminalde tur raporu
7	<code>top_tb</code> , testbench'lerin tamamlanması, bitstream	Teslim paketi hazır

Her şeyi paralel yazıp sonda birleştirmeyin. Adım 4'e kadar oynanabilir bir çekirdek çıkarın, kurallar ve UART onun üstüne binsin. Ters sırada yaparsanız entegrasyonda batarsınız.

9. Vivado iş akışı

İş	Nasıl
Dosya ekleme	Add Sources → Add or create design/simulation sources → Add Files. "Copy sources into project" işaretsiz .
Simülasyon top'u değiştirme	Testbench'e sağ tıkla → Set as Top . Her modülü test ederken değiştireceksiniz.
Simülasyon çalıştırma	Flow Navigator → Run Simulation → Run Behavioral Simulation
Soluk görünen dosyalar	<code>top.v</code> henüz onları örneklemediği için Vivado "disabled" işaretliyor. Bağladıkça düzelir, hata değil.
Sentez / bitstream	Run Synthesis → Run Implementation → Generate Bitstream → Open Hardware Manager → Program Device
Terminal	PuTTY veya Tera Term, Device Manager'dan COM portunu bul, 9600-8-N-1

10. Teslim ve demo

Teslim edilecekler

- Tüm Verilog kaynak dosyaları
- Testbench dosyaları
- Basys3 constraint dosyası (`Basys3_Refleks.xdc`)
- Bitstream dosyası veya üretme adımları

1 sayfalık tanıtım yazısı gönüllü. Switch/LED eşlemesi ya orada ya `top.v`'nin başında comment olarak **zorunlu**.

Demoya hazırlık — her grup üyesi bilmeli

- Kendi modülünün nasıl çalıştığı ve **neden o tasarım kararının alındığı**
- Kendi modülünün diğerleriyle hangi sinyallerle konuştuğu
- Genel oyun akışı ve FSM mantığı
- UART, 7-segment, LED, buton ve switch'lerin projede ne amaçla kullanıldığı

Demoda en çok sorulan tip: "Neden böyle yaptınız?" — "Çalıştı" cevabı yeterli değil. Bölüm 6'daki kuralların her birinin bir gerekçesi var; onları bilmek yeterli.

Akademik dürüstlük

Teslim edilen kod, tasarım ve açıklamalar size ait olmalı. LLM araçları öğrenmek, fikir almak, hata ayıklamak ve kavram anlamak için kullanılabilir; çıktısını doğrudan teslim etmek ihlaldir. Teslimler açık kaynak kodlarla, LLM çıktılarıyla ve diğer grupların teslimleriyle karşılaştırılabilir.